



DevOps Engineer – Docker Responsibilities (Daily, Weekly, Monthly)



Daily Tasks (Everyday Responsibilities)

1. Monitor Running Containers

- Use `docker ps` to verify container status.
- Ensure application containers are in running state.

2. Check Docker Logs

- Analyze logs for errors, crashes, or unexpected restarts.
- Investigate logs immediately if any container restarts.

3. Validate CI/CD Docker Builds

- Check Jenkins (or CI tool) to ensure Docker builds are successful.
- Confirm proper tagging (e.g., versioned, not `latest`).

4. Monitor Image Size and Layers

- Identify sudden increase in image size.
- Ensure proper base images are being used.

5. Docker Daemon Health Check

- Verify that Docker service is active.
- Monitor CPU/memory usage spikes at the host level.

6. Container Cleanup (Optional)

- Remove stopped containers and dangling images to free up disk space.
-



Weekly Tasks (Periodic Responsibilities)

1. Review & Optimize Dockerfiles

- Review Dockerfiles created by dev teams.
- Remove unnecessary layers and implement best practices.

2. Audit Base Images

- Check for known vulnerabilities in commonly used base images.
- Promote use of lightweight and secure images.

3. Validate Docker Registry Images

- Verify that images in DockerHub, ECR, etc., are valid.
- Ensure `latest` tag isn't accidentally overwritten.

4. Enforce Tagging & Versioning Standards

- Make sure versioned tags are used (e.g., `v1.2.3`).
- Avoid using `latest` tag in builds and deployments.

5. Review Docker Compose Files

- Validate structure of `docker-compose.yml` files used in staging or local environments.
- Ensure no port conflicts exist.

6. Test Container Networking

- Verify container-to-container communication (e.g., app ↔ DB).



Monthly Tasks (Strategic Responsibilities)

1. Run Image Vulnerability Scans

- Use tools like Snyk, Trivy, or Docker Scan to detect security issues.
- Share reports with development teams.

2. Clean Up Docker Registry

- Remove unused or outdated images from registries (ECR, GCR).
- Optimize storage and reduce costs.

3. Train Developers on Containerization

- Conduct sessions on Docker basics, caching, and best practices.
- Share tips for writing efficient Dockerfiles.

4. Benchmark Container Performance

- Analyze memory and CPU usage across containers.
- Suggest improvements or tuning.

5. Test Backup & Restore for Container Volumes

- Validate backup strategies for volumes (named or mounted).
- Ensure data recovery readiness.

6. Review Container-Related Incidents

- Analyze recent issues (crashes, slowness, port errors, image mismatch).
- Document root causes and implement permanent fixes.